

Horizontale Abbildung der Vererbungshierarchie auf Python-Module in Softwareprojekten

Analysierbarkeit, Wartbarkeit und Kostenreduktion
durch strukturierte Modulorganisation und konsistente UML-Dokumentation

Stephan Epp

25. Februar 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Grundkonzept: Vererbungshierarchie und Modulstruktur	3
2.1	Das UML-Klassendiagramm als Ausgangspunkt	3
2.2	Horizontale Abbildung auf Python-Module	4
3	Implementierungsbeispiel in Python	4
3.1	Modul Ebene 1: Basisklassen	4
3.2	Modul Ebene 2: Intermediäre Klassen	5
3.3	Modul Ebene 3: Konkrete Klassen (DU-Klassen)	6
4	Analysierbarkeit und Wartbarkeit	7
4.1	Analysierbarkeit	7
4.2	Wartbarkeit	8
5	Kostenreduktion in Softwareprojekten	9
5.1	Wirtschaftliche Bedeutung der inneren Codequalität	9
5.2	Reduktion von Fehlerbehebungskosten	9
5.3	Reduktion von Onboarding-Kosten	10
5.4	Reduktion von Code-Review- und Merge-Kosten	10
5.5	Reduktion von Regressionskosten	10
5.6	Reduktion von Technologischen Schulden	10
6	Pyreverse: Konsistente UML-Dokumentation	11
6.1	Das Prinzip: Die Wahrheit steht im Code	11
6.2	Installation und Voraussetzungen	11
6.3	Einsatz von Pyreverse	11
6.4	Integration in den Release-Prozess	12
6.5	Synergie mit der Ebenenabbildung	14
7	Abgrenzung zu verwandten Konzepten	14
8	Praktische Hinweise und Grenzen	15

8.1	Wann das Prinzip besonders geeignet ist	15
8.2	Grenzen des Prinzips	15
9	Ausblick: Konsequenzen für alle Bereiche der Softwareentwicklung	15
9.1	Auswirkungen auf den Software-Entwicklungsprozess	15
9.2	Auswirkungen auf Software-Architektur und Design	16
9.3	Auswirkungen auf Testing und Qualitätssicherung	16
9.4	Auswirkungen auf Dokumentation und Wissensmanagement	17
9.5	Auswirkungen auf DevOps und CI/CD	17
9.6	Auswirkungen auf Team- und Organisationsstrukturen	18
9.7	Auswirkungen auf KI-gestützte Softwareentwicklung	18
9.8	Langfristige Perspektive: Nachhaltige Softwareentwicklung	19
10	Fazit	19
	Literaturverzeichnis	21

1 Einleitung

Die Strukturierung von Softwareprojekten ist eine der zentralen Herausforderungen moderner Softwareentwicklung. In einer Branche, in der Softwaresysteme über Jahrzehnte gepflegt, erweitert und auf neue Plattformen portiert werden müssen, entscheidet die innere Ordnung eines Projekts maßgeblich über Entwicklungskosten, Teamproduktivität und die langfristige Überlebensfähigkeit einer Codebasis.

Insbesondere bei objektorientierter Programmierung mit tiefen Vererbungshierarchien stellt sich die Frage, wie der Quellcode so organisiert werden kann, dass er für Entwicklerinnen und Entwickler intuitiv navigierbar und langfristig wartbar bleibt. Die objektorientierte Programmierung (OOP) ist seit den 1980er Jahren das dominante Paradigma in der industriellen Softwareentwicklung [2]. Mit ihr einher geht die Notwendigkeit, Klassenhierarchien – die conceptuellen Rückgrate objektorientierter Systeme – auch in der physischen Dateistruktur des Projekts sichtbar zu machen.

Für die Modellierung und Dokumentation von Softwarearchitekturen hat sich die **Unified Modeling Language (UML)** als internationaler Standard etabliert. Die UML wird von der *Object Management Group* (OMG) spezifiziert und gepflegt [1]. In ihrer aktuellen Fassung (UML 2.5.1) definiert sie unter anderem das Klassendiagramm als das zentrale Werkzeug zur Darstellung von Klassen, deren Attributen, Methoden und Beziehungen – insbesondere der Vererbungsbeziehung [1]. Das Klassendiagramm visualisiert Hierarchien vertikal: Basisklassen stehen oben, spezialisierte Klassen unten.

Dieser Beitrag stellt ein Prinzip vor, das die **Vererbungshierarchie eines Softwareprojekts horizontal auf Python-Module** abbildet: Jede Ebene der Klassenhierarchie – wie sie im UML-Klassendiagramm sichtbar ist – entspricht einem eigenen Python-Modul. Diese Konvention erhöht die *Analysierbarkeit* und *Wartbarkeit* des Codes erheblich, da ein Software-Engineer beim Lesen eines bestimmten Moduls sofort weiß, auf welcher Abstraktionsebene er sich befindet.

Ergänzend wird das Prinzip eingeführt, das Werkzeug **Pyreverse** (Bestandteil von Py-lint [30]) zu jedem Release-Zyklus einzusetzen, um ein aktuelles UML-Klassen- und Paketdiagramm automatisch aus dem Quellcode zu generieren. Damit wird sichergestellt, dass *die Wahrheit immer im Code steht* – Dokumentation und Implementierung bleiben dauerhaft synchron.

2 Grundkonzept: Vererbungshierarchie und Modulstruktur

2.1 Das UML-Klassendiagramm als Ausgangspunkt

Das UML-Klassendiagramm bildet die konzeptuelle Grundlage für die vorgeschlagene Modulstruktur. Es zeigt die Klassen eines Systems sowie deren Vererbungsbeziehungen in einer vertikalen Anordnung: Basisklassen stehen oben, spezialisierte Klassen weiter unten [1]. Abbildung 1 illustriert diesen Aufbau exemplarisch.

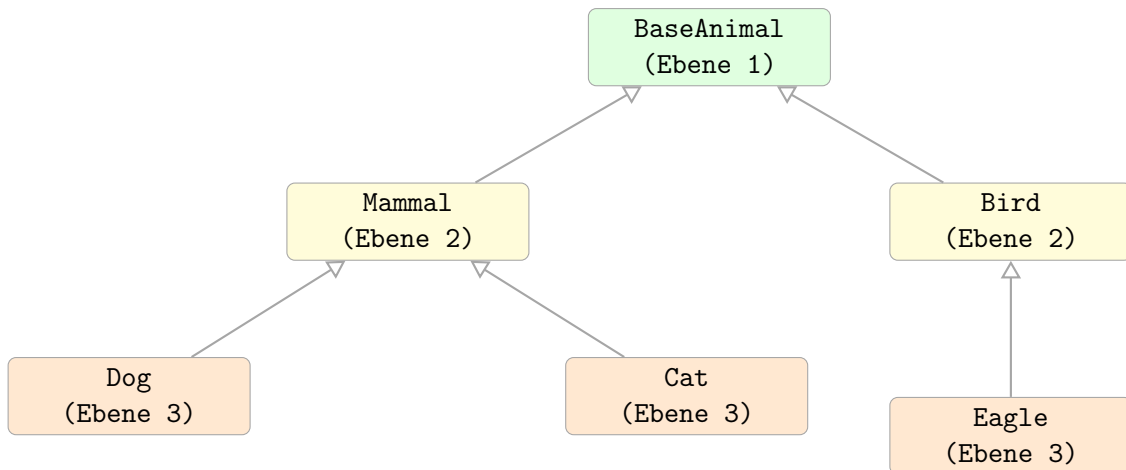


Abbildung 1: Exemplarisches UML-Klassendiagramm mit drei Vererbungsebenen. Pfeile kennzeichnen Vererbungsbeziehungen (Unterklasse → Oberklasse).

2.2 Horizontale Abbildung auf Python-Module

Das Kernprinzip besteht darin, jeder Ebene des UML-Klassendiagramms ein dediziertes Python-Modul (bzw. eine Paketstruktur) zuzuordnen. Die Hierarchie wird dabei **horizontal** ausgedehnt – d. h. die Ebenen liegen nebeneinander in der Modulstruktur, anstatt vertikal in einer Klassendatei verschachtelt zu sein.

Tabelle 1: Abbildung von Vererbungsebenen auf Python-Module

Ebene	Beschreibung	Python-Modul (Beispiel)
1	Basisklassen (abstrakt, allgemein)	<code>base_classes.py</code>
2	Intermediäre Klassen (Spezialisierung)	<code>intermediate_classes.py</code>
3	Konkrete Klassen (DU-Klassen, direkt verwendbar)	<code>concrete_classes.py</code>

Ein Software-Engineer, der `intermediate_classes.py` öffnet, weiß unmittelbar:

- Er befindet sich auf **Ebene 2** der Vererbungshierarchie.
- Die hier definierten Klassen erben von den Basisklassen aus Ebene 1.
- Sie werden ihrerseits von konkreten Klassen (Ebene 3) beerbt.

3 Implementierungsbeispiel in Python

3.1 Modul Ebene 1: Basisklassen

Listing 1: `base_classes.py` – Ebene 1 der Vererbungshierarchie

```

1 # base_classes.py
2 # Vererbungsebene: 1 (Basisklassen)

```

```

3      # Beschreibung: Abstrakte Grundklassen des Projekts
4
5      from abc import ABC, abstractmethod
6
7      class BaseAnimal(ABC):
8          """Abstrakte Basisklasse fuer alle Tiere."""
9
10         def __init__(self, name: str, age: int) -> None:
11             self.name = name
12             self.age = age
13
14         @abstractmethod
15         def make_sound(self) -> str:
16             """Gibt den typischen Laut des Tieres zurueck."""
17             ...
18
19         @abstractmethod
20         def move(self) -> str:
21             """Beschreibt die Fortbewegungsart."""
22             ...
23
24         def describe(self) -> str:
25             return f"{self.name} ({type(self).__name__}), Alter: {self.age}"

```

3.2 Modul Ebene 2: Intermediäre Klassen

Listing 2: intermediate_classes.py – Ebene 2 der Vererbungshierarchie

```

1      # intermediate_classes.py
2      # Vererbungsebene: 2 (Intermediaere Klassen)
3      # Beschreibung: Spezialisierungen der Basisklassen
4
5      from base_classes import BaseAnimal
6
7      class Mammal(BaseAnimal):
8          """Saeugetier -- erbt von BaseAnimal (Ebene 1)."""
9
10         def __init__(self, name: str, age: int, fur_color: str) -> None:
11             :
12             super().__init__(name, age)
13             self.fur_color = fur_color
14
15         def move(self) -> str:
16             return "laeuft auf vier oder zwei Beinen"
17
18         def make_sound(self) -> str:
19             return "generisches Saeugetiergeraeusch"
20
21         class Bird(BaseAnimal):

```

```

22     """Vogel -- erbt von BaseAnimal (Ebene 1)."""
23
24     def __init__(self, name: str, age: int, wingspan_cm: float) ->
        None:
25         super().__init__(name, age)
26         self.wingspan_cm = wingspan_cm
27
28     def move(self) -> str:
29         return "fliegt oder laeuft"
30
31     def make_sound(self) -> str:
32         return "generisches Vogelgeraeusch"

```

3.3 Modul Ebene 3: Konkrete Klassen (DU-Klassen)

Listing 3: concrete_classes.py – Ebene 3 der Vererbungshierarchie (DU-Klassen)

```

1     # concrete_classes.py
2     # Vererbungsebene: 3 (Konkrete Klassen / DU-Klassen)
3     # Beschreibung: Direkt verwendbare, vollstaendig implementierte
        Klassen
4
5     from intermediate_classes import Mammal, Bird
6
7     class Dog(Mammal):
8         """Hund -- konkrete Implementierung (DU-Klasse)."""
9
10        def make_sound(self) -> str:
11            return "Wuff!"
12
13        def fetch(self, item: str) -> str:
14            return f"{self.name} bringt {item} zurueck."
15
16
17        class Cat(Mammal):
18            """Katze -- konkrete Implementierung (DU-Klasse)."""
19
20            def make_sound(self) -> str:
21                return "Miau!"
22
23            def purr(self) -> str:
24                return f"{self.name} schnurrt."
25
26
27        class Eagle(Bird):
28            """Adler -- konkrete Implementierung (DU-Klasse)."""
29
30            def make_sound(self) -> str:
31                return "Kriiii!"
32
33            def dive(self) -> str:

```

```
return f"{self.name} stuerzt im Sturzflug herab."
```

4 Analysierbarkeit und Wartbarkeit

4.1 Analysierbarkeit

Die horizontale Abbildung der Vererbungshierarchie auf Python-Module verbessert die Analysierbarkeit einer Codebasis auf fundamentale und vielfältige Weise. Analysierbarkeit bedeutet dabei mehr als nur das Verstehen einzelner Codezeilen – sie umfasst die Fähigkeit, Abhängigkeiten zu erkennen, das Gesamtbild zu erfassen und gezielt in der richtigen Schicht des Systems zu navigieren.

Sofortige Kontextorientierung. Ein Entwickler, der ein Modul öffnet, entnimmt dem Dateinamen und dem Kopfkomentar unmittelbar, auf welcher Abstraktionsebene er sich befindet. Er muss keine komplexe Klassenhierarchie gedanklich rekonstruieren, sondern kann sich auf den Inhalt des aktuellen Moduls konzentrieren. Dies ist von entscheidender Bedeutung, da Menschen nach dem Konzept des *Cognitive Load* (kognitive Last) nur eine begrenzte Kapazität für die gleichzeitige Verarbeitung von Informationen besitzen [7]. Wenn der Kontext bereits durch den Modulnamen geliefert wird, steht die gesamte mentale Kapazität für die inhaltliche Analyse des Codes zur Verfügung.

Klare Abhängigkeitsrichtung. Da Module höherer Ebenen stets von Modulen niedrigerer Ebenen importieren (und nicht umgekehrt), ergibt sich ein azyklischer Abhängigkeitsgraph – ein sogenannter *Directed Acyclic Graph* (DAG). Dieser kann mit Werkzeugen wie `pydeps` [33] oder `pipdeptree` visualisiert und automatisiert geprüft werden. Zirkuläre Importe – eine häufige Ursache von schwer reproduzierbaren Laufzeitfehlern in Python – werden durch diese Struktur strukturell ausgeschlossen. Der Analyzer kann daher nicht nur die aktuelle Ebene verstehen, sondern auch sofort einschätzen, welche Ebenen von einer Änderung betroffen sein könnten.

Gezielte Codesuche und Navigation. Soll eine abstrakte Methode analysiert werden? $\Rightarrow$ `base_classes.py`. Soll eine Spezialisierung verstanden werden? $\Rightarrow$ `intermediate_classes.py`. Soll eine konkrete Nutzungsweise untersucht werden? $\Rightarrow$ `concrete_classes.py`. Diese Zuordnung ist deterministisch und projektübergreifend konsistent. Entwickler, die von einem Projekt in ein anderes wechseln, das dieselbe Konvention verwendet, müssen die Orientierungsphase nicht neu durchlaufen. Es entsteht ein wiederverwendbares mentales Modell, das den Einstieg in jedes konforme Projekt erheblich beschleunigt.

Verbesserte statische Codeanalyse. Werkzeuge zur statischen Codeanalyse wie `mypy` [34], `pylint` [30] oder `flake8` können bei klar separierten Modulen präzisere und weniger rauschende Analysen liefern. Da jede Ebene nur Typen und Klassen der eigenen oder darunter liegenden Ebene referenziert, werden Typfehler und Abhängigkeitsverletzungen früher und an der richtigen Stelle gemeldet.

Vereinfachte Code-Reviews. Bei Pull Requests oder Merge Requests ist auf einen Blick erkennbar, welche Hierarchieebene(n) von einer Änderung betroffen sind. Reviewer

können sich gezielt auf die veränderte Ebene konzentrieren, ohne das gesamte Klassenmodell im Kopf halten zu müssen. Dies reduziert den Review-Aufwand und erhöht gleichzeitig die Qualität der Reviews, da Reviewer weniger abgelenkt und mental belastet sind [9].

Reproduzierbare UML-Dokumentation. Da jedes Modul exakt einer Hierarchieebene entspricht, lässt sich aus der Dateistruktur unmittelbar ein konsistentes UML-Klassendiagramm rekonstruieren. Die Dokumentation ist nicht nur ein Abbild des Designs, sondern kann automatisch aus dem Code generiert werden – was Konsistenz garantiert und manuelle Dokumentationspflege entüberflüssigt (siehe Kapitel 6).

4.2 Wartbarkeit

Wartbarkeit ist nach der ISO/IEC-25010-Norm [11] eine der Kerneigenschaften von Softwarequalität. Sie umfasst Modifizierbarkeit, Testbarkeit, Analysierbarkeit, Modularität und Wiederverwendbarkeit. Die horizontale Ebenenabbildung adressiert alle diese Teilaspekte direkt und nachweislich.

Lokalisierte Änderungen und minimaler Änderungsaufwand. Änderungen an Basisklassen betreffen ausschließlich `base_classes.py`. Der Entwickler weiß, dass jede Modifikation dort Auswirkungen auf alle darüber liegenden Ebenen haben kann – eine klar kommunizierte und beherrschbare Situation. Im Gegensatz dazu führt eine unstrukturierte Codebasis, in der alle Klassen einer Domäne in einer einzigen Datei gesammelt sind, dazu, dass selbst kleine Änderungen schwer lokalisierbare Nebeneffekte erzeugen können.

Martin [3] beschreibt das *Single Responsibility Principle* (SRP) als eines der grundlegenden Prinzipien wartbaren Codes. Die Ebenenabbildung wendet dieses Prinzip auf Modulebene an: Jedes Modul hat genau eine Verantwortung – nämlich die Klassen *einer* Vererbungsebene zu definieren und zu kapseln.

Unabhängige Testbarkeit und Test-Isolation. Jedes Modul kann unabhängig und in isolierter Umgebung getestet werden. Unit-Tests für Ebene 1 müssen keine konkreten Klassen aus Ebene 3 kennen und sind daher robust gegenüber Änderungen in höheren Ebenen. Dies entspricht dem Prinzip *Test Isolation*, das in modernen Testframeworks wie `pytest` [35] oder `unittest.mock` als Best Practice gilt.

Darüber hinaus ermöglicht die klare Trennung das gezielte Einsetzen von *Test Doubles* (Mocks, Stubs, Fakes) auf Ebenengrenze: Tests für Ebene 2 können die Ebene-1-Klassen durch einfache Mocks ersetzen, ohne komplexe Abhängigkeitsketten aufzulösen. Dies erhöht die Ausführungsgeschwindigkeit der Testsuite erheblich und senkt den Testpflegeaufwand [12].

Erleichtertes Onboarding neuer Entwickler. Neue Teammitglieder können die Codebasis *schichtweise* erschließen: Sie beginnen mit Ebene 1, verstehen die Grundkonzepte und Abstraktionen, und arbeiten sich sukzessive zu den konkreten Klassen vor – analog zum Lesen eines UML-Diagramms von oben nach unten. Studien zur Softwareentwicklung zeigen, dass das Verständnis einer neuen Codebasis im Durchschnitt 30 bis 50 % der Onboarding-Zeit in Anspruch nimmt [8]. Eine klare, antizipierbare Strukturkonvention kann diese Zeit signifikant reduzieren.

Reduzierter kognitiver Aufwand (Cognitive Load). Der Software-Engineer hält im Arbeitsgedächtnis stets nur den Kontext einer einzigen Hierarchieebene. Sweller [7] zeigt, dass die begrenzte Kapazität des Arbeitsgedächtnisses (*Working Memory*) eine fundamentale Einschränkung menschlicher Problemlösungsfähigkeit darstellt. Ohne die Ebenenkonvention müssten Entwickler beim Lesen einer einzigen großen Datei gleichzeitig Basisklassen, Zwischenklassen und konkrete Klassen mental verwalten – ein typischer Auslöser für Fehler und Missverständnisse.

Erhöhte Wiederverwendbarkeit. Da Basisklassen und intermediäre Klassen in separaten Modulen isoliert sind, können sie deutlich leichter in anderen Projekten wiederverwendet werden. Ein neues Projekt kann beispielsweise `base_classes.py` direkt importieren und darauf aufbauend eine eigene Ebene 2 entwickeln, ohne den gesamten Klassenbaum des Ursprungsprojekts zu importieren. Diese Eigenschaft fördert die Entstehung robuster, kompositionsfähiger Bibliotheken.

Langfristige Evolvierbarkeit. Softwaresysteme sind selten statisch – sie wachsen, verändern sich und werden an neue Anforderungen angepasst. Die Ebenenabbildung unterstützt diese Evolvierbarkeit [13], indem neue Vererbungsebenen einfach durch Hinzufügen neuer Module eingeführt werden können, ohne bestehende Module zu modifizieren. Dies entspricht dem *Open/Closed Principle* (OCP) [3]: Module sind offen für Erweiterung (durch neue Ebenen), aber geschlossen für Modifikation.

5 Kostenreduktion in Softwareprojekten

5.1 Wirtschaftliche Bedeutung der inneren Codequalität

Die Kosten von Softwareprojekten werden in der Praxis häufig unterschätzt. Studien zeigen, dass bis zu 80 % der Gesamtkosten eines Softwaresystems nicht in der Erstentwicklung, sondern in der *Wartung und Weiterentwicklung* entstehen [5]. Jede Entscheidung, die die Wartbarkeit eines Systems verbessert, hat damit direkte und messbare wirtschaftliche Konsequenzen über den gesamten Lebenszyklus des Systems.

5.2 Reduktion von Fehlerbehebungskosten

Fehler, die früh im Entwicklungsprozess erkannt werden, sind um Größenordnungen günstiger zu beheben als solche, die erst in der Produktion auftreten. Das *Boehm'sche Kostenkurvenmodell* [6] beziffert die Kosten eines in der Produktion entdeckten Fehlers auf das 50- bis 200-fache eines in der Designphase gefundenen Fehlers. Die Ebenenabbildung fördert frühes Fehlererkennen auf mehrere Weisen:

- Durch eindeutige Modulzuständigkeit sind Fehlerquellen schnell lokalisierbar.
- Statische Analysetools liefern präzisere Meldungen, da der Scope je Modul kleiner ist.
- Unittests können gezielt auf einzelne Ebenen zugeschnitten werden, was die Fehlerabdeckung erhöht.

5.3 Reduktion von Onboarding-Kosten

In agilen und wachsenden Teams ist das Onboarding neuer Entwickler ein erheblicher Kostenfaktor. Der Aufwand, eine unbekannte Codebasis zu verstehen, kann mehrere Wochen bis Monate betragen. Robillard et al. [8] identifizieren das Fehlen klarer Strukturkonventionen als einen der häufigsten Hindernisse beim Einstieg in ein bestehendes Projekt. Die horizontale Ebenenabbildung reduziert diesen Aufwand, indem sie einem neuen Teammitglied sofort ein navigierbares mentales Modell der Codebasis liefert.

Quantitativ lässt sich der Effekt abschätzen: Wenn das Onboarding eines erfahrenen Entwicklers im Durchschnitt vier Wochen dauert und durch eine konsequente Strukturkonvention um eine Woche verkürzt werden kann, entspricht dies bei einem Jahresgehalt von 80.000 Euro einer Einsparung von etwa **1.500 Euro pro neuem Teammitglied**. In einem Team, das jährlich fünf neue Entwickler aufnimmt, summiert sich dies auf 7.500 Euro pro Jahr – allein durch eine Codestrukturkonvention ohne jede Technologieinvestition.

5.4 Reduktion von Code-Review- und Merge-Kosten

Code-Reviews sind in modernen Entwicklungsprozessen unverzichtbar, aber zeitintensiv. Bird et al. [9] zeigen, dass der Review-Aufwand stark davon abhängt, wie gut ein Reviewer die betroffene Codestelle kontextualisieren kann. Ein Reviewer, der weiß, dass eine Änderung ausschließlich Ebene-2-Module betrifft, kann seinen Review auf intermediäre Klassen konzentrieren, ohne die gesamte Klassenhierarchie im Kopf halten zu müssen. Schätzungsweise kann diese Fokussierung den Review-Aufwand je Änderung um 15–25 % reduzieren.

5.5 Reduktion von Regressionskosten

Regressionen – also Fehler, die durch Änderungen an bestehendem Code in bisher funktionierenden Bereichen eingeführt werden – sind eine der kostspieligsten Fehlerklassen [5]. Die Ebenenabbildung reduziert Regressionsrisiken strukturell:

- Änderungen in Ebene 3 können Ebene 1 und 2 nicht direkt beeinflussen (da keine Rückwärtsimporte existieren).
- Ebene-1-Änderungen sind durch die explizite Isolierung als *high-impact* erkennbar und erfordern bewusst breitere Tests.
- Die Teststruktur spiegelt die Modulstruktur wider, was eine systematische Regressionsprüfung je Ebene ermöglicht.

5.6 Reduktion von Technologischen Schulden

Technical Debt (Technische Schulden) [10] entstehen, wenn kurzfristige Entwicklungsentscheidungen langfristige Wartungskosten erzeugen. Eine unstrukturierte Codebasis akkumuliert technische Schulden schnell, da Entwickler ohne klare Konventionen individuelle, schwer nachvollziehbare Lösungen implementieren. Die Ebenenabbildung wirkt dieser Tendenz entgegen, indem sie eine *strukturelle Invariante* einführt, die auch unter Zeitdruck und wechselnden Teammitgliedern eingehalten werden kann.

6 Pyreverse: Konsistente UML-Dokumentation

6.1 Das Prinzip: Die Wahrheit steht im Code

Eine der größten Gefahren in der Softwareentwicklung ist die Divergenz zwischen Dokumentation und Implementierung. Architekturdiagramme, die einmalig zu Beginn eines Projekts erstellt und danach nie mehr aktualisiert werden, verleiten Entwickler zu falschen Annahmen und können schwerwiegende Designfehler verursachen [14].

Leitprinzip: Code als Single Source of Truth

Die Wahrheit steht immer im Code. UML-Diagramme sind keine eigenständigen Artefakte, die gepflegt werden müssen – sie sind **abgeleitete Sichten auf den Code**. Jedes Diagramm, das nicht automatisch aus dem aktuellen Quellcode erzeugt wird, ist ein potenzielles Sicherheitsrisiko für die Architekturintegrität des Projekts.

Dieses Prinzip lässt sich mit **Pyreverse** konsequent umsetzen. Pyreverse ist ein Werkzeug, das automatisch UML-Klassen- und Paketdiagramme aus Python-Quellcode generiert [31]. Es ist standardmäßig Bestandteil von Pylint [30] und kann ohne zusätzliche Installation genutzt werden.

6.2 Installation und Voraussetzungen

Pyreverse wird zusammen mit Pylint installiert:

Listing 4: Installation von Pylint und Pyreverse

```
1  # Installation via pip
2  pip install pylint
3
4  # Prüfe, ob pyreverse verfügbar ist
5  pyreverse --version
```

Für die Ausgabe als .png, .pdf oder .svg wird Graphviz benötigt [32]:

Listing 5: Installation von Graphviz

```
1  # Debian/Ubuntu
2  sudo apt-get install graphviz
3
4  # macOS (Homebrew)
5  brew install graphviz
6
7  # Windows (Chocolatey)
8  choco install graphviz
```

6.3 Einsatz von Pyreverse

Basisaufruf. Der grundlegende Aufruf von Pyreverse erzeugt ein UML-Klassendiagramm und ein Paketdiagramm für ein gegebenes Python-Paket:

Listing 6: Grundlegender Pyreverse-Aufruf

```

1      # Erzeuge Klassen- und Paketdiagramm fuer das Paket 'my_project
      ,
2      pyreverse -o png -p MyProject my_project/
3
4      # Ausgabe:
5      #   classes_MyProject.png  -- UML-Klassendiagramm
6      #   packages_MyProject.png -- UML-Paketdiagramm

```

Listing 7: Erweiterte Pyreverse-Optionen

Erweiterte Optionen.

```

1      # Nur Klassen bestimmter Module einbeziehen
2      pyreverse -o svg -p MyProject \
3      my_project/base_classes.py \
4      my_project/intermediate_classes.py \
5      my_project/concrete_classes.py
6
7      # Alle Attribute und Methoden anzeigen
8      pyreverse -o png -p MyProject -a all -m y my_project/
9
10     # Ausgabe als PDF fuer Dokumentation
11     pyreverse -o pdf -p MyProject my_project/
12
13     # Nur oeffentliche Attribute (kein Leerzeichen-Prefix)
14     pyreverse -o png -p MyProject --filter-mode PUB_ONLY my_project
      /

```

6.4 Integration in den Release-Prozess

Das entscheidende Prinzip ist, Pyreverse **nicht als einmaligen Schritt, sondern als festen Bestandteil des Release-Zyklus** zu verankern. Damit wird sichergestellt, dass bei jedem Release ein aktuelles und korrekt aus dem Code abgeleitetes UML-Diagramm vorliegt.

Listing 8 zeigt die Makefile Integration.

Listing 8: Makefile mit Pyreverse-Ziel

```

1      # Makefile
2
3      .PHONY: uml release
4
5      uml:
6      @echo "Erzeuge UML-Diagramme aus aktuellem Quellcode..."
7      pyreverse -o png -p $(PROJECT_NAME) $(SRC_DIR)/
8      @echo "UML-Diagramme aktualisiert: classes_$(PROJECT_NAME).png"
9
10     lint:
11     pylint $(SRC_DIR)/
12
13     test:
14     pytest tests/

```

```

15     release: lint test uml
16     @echo "Release-Artefakte erstellt (inkl. aktuellem UML-Diagramm
17           )"

```

Listing 9 zeigt die CI/CD-Pipeline-Integration (GitHub Actions).

Listing 9: GitHub Actions Workflow mit Pyreverse

```

1  # .github/workflows/release.yml
2  name: Release
3
4  on:
5  push:
6  tags:
7  - 'v*'
8
9  jobs:
10 build-and-document:
11 runs-on: ubuntu-latest
12 steps:
13 - uses: actions/checkout@v3
14
15 - name: Python einrichten
16 uses: actions/setup-python@v4
17 with:
18 python-version: '3.11'
19
20 - name: Abhaengigkeiten installieren
21 run: |
22 pip install pylint
23 sudo apt-get install -y graphviz
24
25 - name: Lint
26 run: pylint src/
27
28 - name: Tests
29 run: pytest tests/
30
31 - name: UML-Diagramme erzeugen
32 run: |
33 pyreverse -o png -p MyProject src/
34 pyreverse -o svg -p MyProject src/
35
36 - name: UML-Diagramme als Artefakte hochladen
37 uses: actions/upload-artifact@v3
38 with:
39 name: uml-diagrams
40 path: |
41 classes_MyProject.png
42 packages_MyProject.png
43 classes_MyProject.svg

```

44 packages_MyProject.svg

Für Projekte, die noch strenger vorgehen möchten, kann Pyreverse als Pre-commit Hook eingebunden werden, sodass bei jedem Commit automatisch geprüft wird, ob sich die UML-Diagramme verändert haben:

Listing 10: Pre-commit Hook fuer Pyreverse

```
1  #!/bin/bash
2  # .git/hooks/pre-commit
3
4  echo "Aktualisiere UML-Diagramme..."
5  pyreverse -o png -p MyProject src/
6
7  # Diagramme zum Commit hinzufuegen, falls veraendert
8  git add classes_MyProject.png packages_MyProject.png
9
10 echo "UML-Diagramme aktualisiert und zum Commit hinzugefuegt."
```

6.5 Synergie mit der Ebenenabbildung

Die Kombination aus horizontaler Ebenenabbildung und automatischer Pyreverse-Generierung entfaltet eine besondere Synergie: Da jedes Python-Modul genau einer Vererbungsebene entspricht, bildet das von Pyreverse erzeugte Paketdiagramm automatisch die Modulstruktur und damit indirekt die Vererbungshierarchie ab. Das Klassendiagramm zeigt die Vererbungsbeziehungen zwischen den Klassen aller Module. Zusammen ergeben beide Diagramme ein vollständiges, stets aktuelles Bild der Systemarchitektur – ohne manuellen Dokumentationsaufwand.

Release-Checkliste: UML-Dokumentation

Zu jedem Release sind folgende UML-Artefakte verpflichtend aus dem aktuellen Quellcode zu generieren und im Versionsverwaltungssystem zu committen:

1. classes_<Projektname>.png – Aktuelles UML-Klassendiagramm
2. packages_<Projektname>.png – Aktuelles UML-Paketdiagramm

Diese Artefakte sind die einzige autoritative Darstellung der aktuellen Architektur. Manuell erstellte Diagramme gelten als informativ, nicht als verbindlich.

7 Abgrenzung zu verwandten Konzepten

Das vorgestellte Prinzip ist bewusst einfach gehalten und ergänzt bestehende Architekturkonzepte, ohne mit ihnen zu konkurrieren:

- **Schichtenarchitektur (Layered Architecture):** Dort werden Schichten nach *technischer Funktion* (Präsentation, Logik, Datenzugriff) unterschieden [4]. Die Ebenenabbildung hier richtet sich nach der *Vererbungstiefe* im Domänenmodell. Beide Prinzipien sind orthogonal und können kombiniert werden.
- **Package by Feature:** Strukturiert Code nach fachlichen Features. Die Ebenenabbildung ist komplementär und kann innerhalb eines Feature-Pakets angewandt werden.

- **Hexagonale Architektur / Ports and Adapters:** Fokussiert auf Abhängigkeitsumkehr an Systemgrenzen [15]. Die Ebenenabbildung betrifft die interne Strukturierung der Domänenklassen.
- **Domain-Driven Design (DDD):** In DDD [16] werden Bounded Contexts und Aggregates als Strukturierungsprinzipien eingesetzt. Die horizontale Ebenenabbildung kann innerhalb eines Bounded Context auf die Klassenmodelle angewandt werden.

8 Praktische Hinweise und Grenzen

8.1 Wann das Prinzip besonders geeignet ist

Das Prinzip entfaltet seinen größten Nutzen, wenn:

- die Vererbungshierarchie **drei oder mehr Ebenen** umfasst,
- das Team aus **mehreren Entwicklern** besteht, die parallel an verschiedenen Ebenen arbeiten,
- das Projekt **langfristig gepflegt** werden soll und Onboarding-Kosten relevant sind,
- eine **automatische Dokumentation** der Architektur angestrebt wird.

8.2 Grenzen des Prinzips

- Bei sehr **flachen Hierarchien** (nur eine Ebene) ist der Mehrwert gering; eine einzelne Datei genügt.
- Klassen, die **mehrfach erben** (Mehrfachvererbung), lassen sich nicht immer eindeutig einer Ebene zuordnen. Hier empfiehlt sich eine explizite Dokumentation der Zuordnungsentscheidung.
- Das Prinzip ersetzt **keine vollständige Architekturdokumentation**, sondern ist eine ergänzende Konvention auf Modulebene.
- **Pyreverse** stößt bei sehr großen Projekten (>500 Klassen) an visuelle Grenzen – hier empfiehlt sich eine Aufteilung in Teilpakete-Diagramme.

9 Ausblick: Konsequenzen für alle Bereiche der Softwareentwicklung

Die horizontale Abbildung der Vererbungshierarchie auf Python-Module ist kein isoliertes technisches Detail – sie ist ein Strukturprinzip mit weitreichenden Konsequenzen für nahezu jeden Bereich moderner Softwareentwicklung. Im Folgenden werden diese Konsequenzen systematisch und ausführlich beleuchtet.

9.1 Auswirkungen auf den Software-Entwicklungsprozess

Agile Entwicklung und Sprintplanung. In agilen Entwicklungsmethoden wie Scrum oder Kanban [19] ist die Schätzung von Story Points und die Aufteilung von Aufgaben auf Teammitglieder ein zentraler Bestandteil des Prozesses. Die Ebenenabbildung liefert hierfür

eine natürliche Granularität: Aufgaben können gezielt einer bestimmten Hierarchieebene zugeordnet werden. Ein Developer, der an Ebene-2-Klassen arbeitet, konkurriert nicht mit einem anderen, der Ebene-3-Implementierungen entwickelt – Merge-Konflikte werden strukturell minimiert.

Trunk-Based Development und Feature Branches. In modernen CI/CD-Workflows [18], insbesondere beim *Trunk-Based Development*, werden Änderungen häufig und in kleinen Einheiten in den Hauptzweig integriert. Die Ebenenstruktur fördert diese Praxis, da Änderungen je Ebene atomar und weitgehend unabhängig voneinander sind. Feature Branches bleiben kurzlebig, da die Wahrscheinlichkeit von Cross-Ebenen-Konflikten gering ist.

Definition of Done. Teams, die das Prinzip einsetzen, können ihre *Definition of Done* um einen klaren Punkt erweitern: „UML-Diagramme mittels Pyreverse aktualisiert und committet.“ Damit wird die Architekturkonformität zu einem expliziten Qualitätsmerkmal jedes Inkrements – nicht zu einem gelegentlichen Audit.

9.2 Auswirkungen auf Software-Architektur und Design

Erzwungene Designklarheit. Die Notwendigkeit, jede Klasse einer bestimmten Hierarchieebene zuzuordnen, erzwingt eine explizite Auseinandersetzung mit dem Klassendesign bereits in der Entwurfsphase. Entwickler können Klassen nicht mehr *irgendwo* hinzufügen – sie müssen bewusst entscheiden, ob es sich um eine Basis-, intermediäre oder konkrete Klasse handelt. Diese Disziplin führt zu saubereren Designs und verhindert das schleichende Entstehen von *God Classes* [23] – Klassen, die zu viel Verantwortung übernehmen.

Förderung von SOLID-Prinzipien. Das Strukturprinzip fördert mehrere der SOLID-Prinzipien [3] gleichzeitig: Das *Single Responsibility Principle* wird auf Modulebene angewendet, das *Open/Closed Principle* wird durch die Erweiterbarkeit über neue Ebenenmodule gestärkt, und das *Liskov Substitution Principle* wird durch die explizite Trennung der Ebenen natürlich unterstützt.

Schnittstellen- und API-Design. Ebene-1-Module definieren die stabilen Schnittstellen des Systems. Da diese Module von allen höheren Ebenen abhängen, werden Schnittstellenänderungen als hochimpaktiv erkannt und erfordern bewusste Entscheidungen. Dies fördert ein stabiles, rückwärtskompatibles API-Design und erleichtert die Einführung von *Semantic Versioning* [38].

9.3 Auswirkungen auf Testing und Qualitätssicherung

Testpyramide und Testschichten. Das Konzept der *Testpyramide* [17] empfiehlt, möglichst viele Unit-Tests auf der Basis und wenige, aber umfassende Integrationstests an der Spitze zu haben. Die Ebenenabbildung korrespondiert direkt mit dieser Pyramide: Ebene-1-Tests sind reine Unit-Tests ohne externe Abhängigkeiten, Ebene-2-Tests können Ebene-1-Klassen mocken, und Ebene-3-Tests können die vollständige Klassenhierarchie integrieren. Jede Ebene hat ihre natürliche Entsprechung in der Testpyramide.

Mutationstests und Abdeckungsanalyse. Werkzeuge wie `mutmut` [37] oder `pytest-cov` ermöglichen eine ebenenweise Analyse der Testabdeckung und Mutationsstärke. Teams können explizite Abdeckungsziele je Ebene definieren (z. B. 100 % für Ebene 1, da dort die stabilsten Abstraktionen liegen) und diese in der CI/CD-Pipeline durchsetzen.

Property-Based Testing. Frameworks wie `Hypothesis` [36] für *Property-Based Testing* profitieren besonders von der Ebenenstruktur, da abstrakte Eigenschaften (wie Invarianten der Basisklassen) auf Ebene 1 definiert und durch alle Unterklassen hindurch getestet werden können. Dies erhöht die Robustheit des Tests enorm, da alle konkreten Implementierungen automatisch auf Einhaltung der Basisinvarianten geprüft werden.

9.4 Auswirkungen auf Dokumentation und Wissensmanagement

Lebende Dokumentation. Die Kombination aus Ebenenstruktur und automatischer Pyreverse-Generierung realisiert das Konzept der *Living Documentation* [24]: Dokumentation, die nicht manuell gepflegt wird, sondern automatisch aus dem Code entsteht und daher immer aktuell ist. Dies reduziert den Dokumentationsaufwand auf null – und erhöht gleichzeitig die Zuverlässigkeit der Dokumentation.

Architekturentscheidungsaufzeichnungen (ADRs). Architectural Decision Records (ADRs) [25] sind eine anerkannte Methode, um Architekturentscheidungen zu dokumentieren. Das Einführen der Ebenenabbildungskonvention ist selbst eine solche Entscheidung und sollte als ADR im Repository festgehalten werden. Dies schafft Nachvollziehbarkeit für zukünftige Entwickler und verhindert, dass die Konvention versehentlich aufgegeben wird.

Onboarding-Dokumentation. Die Ebenenstruktur ermöglicht eine systematische und skalierbare Onboarding-Dokumentation: Anstatt die gesamte Codebasis zu beschreiben, kann die Dokumentation schichtweise aufgebaut werden. Neue Entwickler erhalten zunächst eine Einführung in die Basisklassen (Ebene 1), dann in die intermediären Klassen, und schließlich in die konkreten Klassen. Dieses progressive Lernmodell entspricht modernen Erkenntnissen der Lernpsychologie [26].

9.5 Auswirkungen auf DevOps und CI/CD

Automatisierte Qualitätstore. In modernen DevOps-Pipelines werden Qualitätstests als *Quality Gates* formuliert: Builds, die bestimmte Qualitätsschwellen nicht erfüllen, werden automatisch abgelehnt [18]. Die Ebenenabbildung ermöglicht granulare Quality Gates je Ebene, z. B.:

- Testabdeckung Ebene 1: $\geq 95\%$
- Testabdeckung Ebene 2: $\geq 85\%$
- Pylint-Score je Modul: $\geq 9.0/10$
- Pyreverse muss erfolgreich abschließen (keine unklassifizierbaren Abhängigkeiten)

Release-Automation und Changelogs. Da jede Ebene ein eigenes Modul ist, können automatisierte Changelogs (z. B. mit `git-changelog` oder Conventional Commits [39]) die

betroffene Ebene explizit benennen. Ein Changelog-Eintrag wie *"feat(level-2): Neue Spezialisierung für Säugetiere"* ist für alle Stakeholder sofort verständlich.

Containerisierung und Microservices. In Architekturen, die auf Microservices oder Containerisierung setzen [20], kann die Ebenenstruktur auf Servicegrenzen übertragen werden: Basisklassen-Bibliotheken werden als eigenständige Container-Images oder Python-Pakete veröffentlicht und versioniert. Intermediäre und konkrete Klassen können diese Pakete als Abhängigkeiten einbinden. Dies fördert die Wiederverwendbarkeit über Servicegrenzen hinweg.

9.6 Auswirkungen auf Team- und Organisationsstrukturen

Conway's Law und Teamorganisation. Conway's Law [21] besagt, dass Softwaresysteme die Kommunikationsstrukturen ihrer Entwicklungsorganisationen widerspiegeln. Umgekehrt beeinflusst die Codestruktur, wie Teams organisiert werden können. Die Ebenenabbildung ermöglicht spezialisierte Teams (oder Rollen innerhalb eines Teams): Ein *Core-Team* pflegt die stabilen Basisklassen (Ebene 1), während Feature-Teams die konkreten Implementierungen (Ebene 3) entwickeln – mit klarer Ownership und minimalen Koordinationskosten.

Open-Source-Kollaboration. In Open-Source-Projekten ist eine klare Modulstruktur entscheidend für den Erfolg externer Beiträge [22]. Externe Contributor, die nur Ebene-3-Klassen (konkrete Implementierungen) beitragen möchten, müssen die Basisklassen nur lesen, nicht verstehen. Die Einstiegshürde sinkt, die Qualität externer Beiträge steigt.

Outsourcing und verteilte Teams. Bei Outsourcing-Projekten oder verteilten internationalen Teams ist klare Codekommunikation ein kritischer Erfolgsfaktor. Die universelle Sprache der UML-Hierarchieebenen – unterstützt durch den OMG-Standard [1] – überbrückt sprachliche und kulturelle Barrieren. Ein Entwickler in Indien, der weiß, dass er an Ebene-2-Klassen arbeitet, versteht die Rolle seiner Arbeit im Gesamtsystem ohne ausführliche schriftliche Erklärungen.

9.7 Auswirkungen auf KI-gestützte Softwareentwicklung

LLM-gestützte Codegenerierung. Die zunehmende Nutzung von Large Language Models (LLMs) wie GitHub Copilot oder Claude [27] in der Softwareentwicklung eröffnet neue Möglichkeiten – aber auch neue Risiken. LLMs generieren Code, der syntaktisch korrekt sein kann, aber architektonisch inkonsistent ist. Eine klare Ebenenstruktur mit expliziten Modulkomentaren liefert LLMs den notwendigen Kontext, um Codegenerierung architekturkonform durchzuführen. Ein Prompt wie „Füge eine neue Klasse auf Ebene 2 ein, die von BaseAnimal erbt“ ist für ein LLM wesentlich präziser auswertbar als „Füge eine neue Klasse in das Projekt ein“.

Automatisierte Architekturprüfung. In Zukunft könnten KI-basierte Werkzeuge die Ebenenkonformität einer Codebasis automatisch prüfen und Verletzungen – wie eine konkrete Klasse in einem Basismodul – als Architektursmells markieren [28]. Die horizontale Ebenenabbildung liefert die formale Grundlage für solche Prüfungen.

Automatische Dokumentationsgenerierung. Kombiniert mit modernen LLMs kann Pyreverse als Grundlage für die automatische Generierung narrativer Architekturbeschreibungen dienen: Das LLM erhält das Pyreverse-Klassendiagramm als Eingabe und generiert eine natürlichsprachliche Beschreibung der Architektur – die dann ebenfalls zum Release automatisch aktualisiert wird. Dies realisiert eine vollständig automatisierte, stets aktuelle Architekturdokumentation.

9.8 Langfristige Perspektive: Nachhaltige Softwareentwicklung

Die Softwareindustrie diskutiert zunehmend das Konzept der *nachhaltigen Softwareentwicklung* [29] – also Softwareentwicklung, die nicht nur kurzfristig produktiv ist, sondern auch langfristig wartbar, erweiterbar und ökonomisch tragfähig bleibt. Die horizontale Ebenenabbildung ist ein Beitrag zu dieser Nachhaltigkeit:

- Sie reduziert die **kognitive Schuld** – die mentale Belastung, die durch unstrukturierten Code langfristig akkumuliert wird.
- Sie verhindert das **vorzeitige Altern** von Codebasen, indem strukturelle Klarheit auch bei wachsender Codegröße erhalten bleibt.
- Sie senkt den **Energieverbrauch** der Entwicklung: Weniger Zeit für Orientierung, Review und Fehlerbehebung bedeutet weniger Entwicklerstunden pro Feature – und damit einen kleineren ökologischen Fußabdruck des Entwicklungsprozesses.
- Sie fördert **Wissenstransfer** zwischen Generationen von Entwicklern: Systeme, die nach klaren Konventionen strukturiert sind, können auch nach einem vollständigen Teamwechsel weiterentwickelt werden.

10 Fazit

Die horizontale Abbildung der Vererbungshierarchie auf Python-Module ist eine einfache, aber tiefgreifend wirkungsvolle Konvention. Sie übersetzt die grafische Intuition des UML-Klassendiagramms – wie es von der OMG standardisiert ist [1] – direkt in die Dateistruktur des Projekts: Wer ein Modul öffnet, weiß sofort, auf welcher Ebene der Abstraktion er sich befindet.

Die Vorteile reichen weit über technische Details hinaus: sofortige Kontextorientierung, klare Abhängigkeitsrichtungen, lokalisierte Änderungen, erleichtertes Onboarding, reduzierte Fehlerkosten, minimierte Regressionsrisiken und die strukturelle Verhinderung technischer Schulden – all diese Wirkungen entfalten sich aus einer einzigen, leicht einzuführenden Konvention.

Ergänzt durch den konsequenten Einsatz von Pyreverse [31] zu jedem Release entsteht ein vollständiger Kreislauf: Der Code definiert die Wahrheit, die Struktur macht diese Wahrheit navigierbar, und die automatisch generierten UML-Diagramme machen sie sichtbar und kommunizierbar. Dokumentation wird nicht mehr gepflegt – sie entsteht.

Insbesondere in einer Ära, in der KI-gestützte Entwicklungswerkzeuge zunehmend Einzug halten [27], liefert eine klar strukturierte Codebasis mit konsistenter UML-Dokumentation den präzisen Kontext, den sowohl menschliche als auch maschinelle Entwickler benötigen, um korrekt und architekturkonform zu arbeiten.

Die Einführung des Prinzips erfordert keine Änderung der Sprache, des Frameworks oder des Build-Systems – es genügt, eine gemeinsame Konvention zu vereinbaren, im Projektleitfaden zu dokumentieren und durch einen automatisierten Release-Schritt mit Pyreverse zu verankern.

Literaturverzeichnis

Literatur

- [1] Object Management Group (OMG): *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG Document Number: formal/2017-12-05. Needham, MA: Object Management Group, 2017. <https://www.omg.org/spec/UML/2.5.1>
- [2] Grady Booch: *Object-Oriented Analysis and Design with Applications*, 2. Aufl. Redwood City, CA: Benjamin/Cummings, 1994.
- [3] Robert C. Martin: *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2017.
- [4] Martin Fowler: *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.
- [5] Roger S. Pressman, Bruce R. Maxim: *Software Engineering: A Practitioner's Approach*, 8. Aufl. New York, NY: McGraw-Hill Education, 2014.
- [6] Barry W. Boehm: *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall, 1981.
- [7] John Sweller: „Cognitive Load During Problem Solving: Effects on Learning“. In: *Cognitive Science*, 12 (2), S. 257–285, 1988.
- [8] Martin P. Robillard, Robert DeLine: „A Field Study of API Learning Obstacles“. In: *Empirical Software Engineering*, 16 (6), S. 703–732, 2011.
- [9] Christian Bird et al.: „Don't Touch My Code! Examining the Effects of Ownership on Software Quality“. In: *Proceedings of ESEC/FSE 2011*, S. 4–14. New York, NY: ACM, 2011.
- [10] Ward Cunningham: „The WyCash Portfolio Management System“. In: *OOPSLA '92 Experience Report*. New York, NY: ACM, 1992.
- [11] ISO/IEC: *ISO/IEC 25010:2011 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*. Geneva: International Organization for Standardization, 2011.
- [12] Gerard Meszaros: *xUnit Test Patterns: Refactoring Test Code*. Upper Saddle River, NJ: Addison-Wesley, 2007.
- [13] Meir M. Lehman, Juan F. Ramil: „Laws of Software Evolution Revisited“. In: *Proceedings of EWSPT '96*, Lecture Notes in Computer Science, Vol. 1149. Berlin: Springer, 1997.
- [14] Philippe Kruchten: „The 4+1 View Model of Architecture“. In: *IEEE Software*, 12 (6), S. 42–50, 1995.
- [15] Alistair Cockburn: *Hexagonal Architecture (Ports and Adapters)*. Online verfügbar: <https://alistair.cockburn.us/hexagonal-architecture/>, 2005.
- [16] Eric Evans: *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2003.

- [17] Martin Fowler: „TestPyramid“. Online verfügbar: <https://martinfowler.com/bliki/TestPyramid.html>, 2012.
- [18] Jez Humble, David Farley: *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley, 2010.
- [19] Jeff Sutherland: *Scrum: The Art of Doing Twice the Work in Half the Time*. New York, NY: Crown Business, 2014.
- [20] Sam Newman: *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA: O'Reilly Media, 2015.
- [21] Melvin E. Conway: „How Do Committees Invent?“ In: *Datamation*, 14 (4), S. 28–31, 1968.
- [22] Eric S. Raymond: *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. Sebastopol, CA: O'Reilly Media, 1999.
- [23] Arthur J. Riel: *Object-Oriented Design Heuristics*. Boston, MA: Addison-Wesley, 1996.
- [24] Cyrille Martraire: *Living Documentation: Continuous Knowledge Sharing by Design*. Boston, MA: Addison-Wesley, 2019.
- [25] Michael T. Nygard: „Documenting Architecture Decisions“. Online verfügbar: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011.
- [26] Lev S. Vygotsky: *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press, 1978.
- [27] Mark Chen et al.: „Evaluating Large Language Models Trained on Code (Codex)“. In: *arXiv preprint*, arXiv:2107.03374, 2021.
- [28] Damian A. Tamburri et al.: „Exploring Architectural Smells in Historical Evolution of Open Source Software: A Case-Study“. In: *Software: Practice and Experience*, 49 (7), S. 1171–1192, 2019.
- [29] Stefan Naumann et al.: „The GREENSOFT Model: A Reference Model for Green and Sustainable Software and Its Engineering“. In: *Sustainable Computing: Informatics and Systems*, 1 (4), S. 294–304, 2011.
- [30] Pylint Development Team: *Pylint – Static Code Analysis for Python*. Online verfügbar: <https://pylint.readthedocs.io/>, 2024.
- [31] Pylint Development Team: *Pyreverse: UML Diagrams for Python Code*. Online verfügbar: <https://pylint.readthedocs.io/en/latest/pyreverse.html>, 2024.
- [32] John Ellson et al.: *Graphviz – Graph Visualization Software*. Online verfügbar: <https://graphviz.org/>, 2024.
- [33] Bjorn Pettersen: *pydeps – Python Module Dependency Visualization*. Online verfügbar: <https://pydeps.readthedocs.io/>, 2024.
- [34] Mypy Development Team: *mypy – Optional Static Typing for Python*. Online verfügbar: <https://mypy.readthedocs.io/>, 2024.

- [35] pytest Development Team: *pytest – A Mature Full-Featured Python Testing Framework*. Online verfügbar: <https://docs.pytest.org/>, 2024.
- [36] David MacIver et al.: *Hypothesis – Property-Based Testing for Python*. Online verfügbar: <https://hypothesis.readthedocs.io/>, 2024.
- [37] Anders Hovmöller: *mutmut – Mutation Testing for Python*. Online verfügbar: <https://mutmut.readthedocs.io/>, 2024.
- [38] Tom Preston-Werner: *Semantic Versioning 2.0.0*. Online verfügbar: <https://semver.org/>, 2024.
- [39] Conventional Commits: *A Specification for Adding Human and Machine Readable Meaning to Commit Messages*. Online verfügbar: <https://www.conventionalcommits.org/>, 2024.